

RAG Study Note

Summary

RAG(Retrieval-Augmented Generation, 검색 증강 생성)는 LLM(Large Language Model)이 답변을 생성하기 전에 external knowledge base에서 relevant information을 retrieve하고, 이를 prompt/context에 include하여 더 grounded 답변을 생성하는 architecture

Content

1. RAG?

- LLM의 intrinsic knowledge(내부 지식)만 사용하는 것이 아니라
- external database(외부 데이터베이스)나 document corpus(문서 집합)에서 query와 관련된 information을 retrieve한 뒤
- 해당 information을 LLM input에 include하여 답변을 생성하는 방식

RAG = LLM intrinsic knowledge + external retrieved knowledge

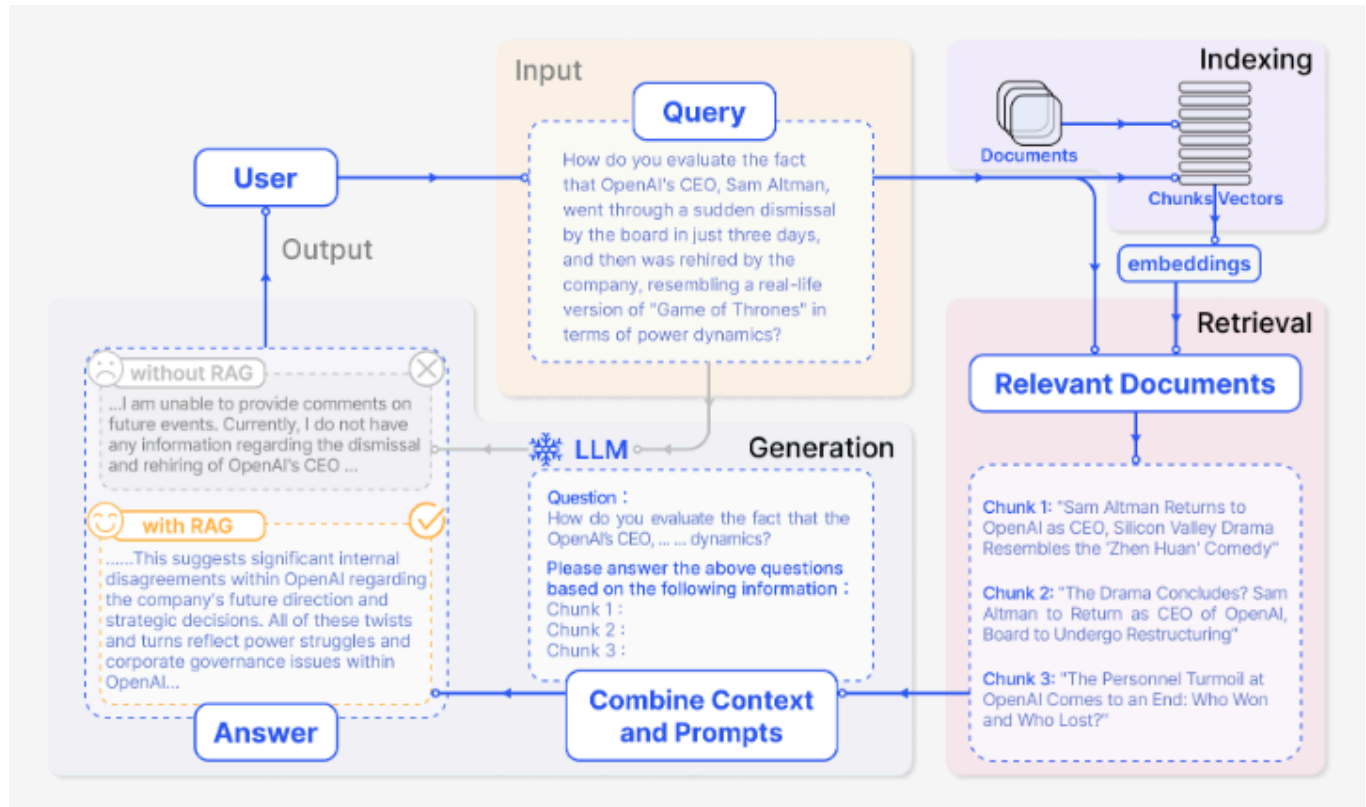
2. RAG의 등장 배경

LLM은 impressive capabilities를 보여주지만, 한계가 존재

- **hallucination**
 - 근거가 없거나 틀린 내용을 그럴듯하게 생성하는 문제
- **outdated knowledge**
 - training data 시점 이후의 latest information이나 변경된 내용을 반영하기 어려운 문제
- **non-transparent reasoning**
 - LLM이 왜 이런 답변을 생성했는지 내부 처리 과정을 확인하기 어려운 문제
 - knowledge가 model parameter안에 compressed되어 있기 때문에 답변 근거가 명확하지 않음
- **untraceable source**
 - answer가 어떤 document나 information에서 나온 것인지 trace하기 어려운 문제

RAG는 model을 매번 retrain하는 방식이 아니라, 답변을 생성하는 시점에 필요한 external knowledge를 retrieve해서 prompt/context로 provide하여, LLM의 한계를 해결

3. RAG의 기본 구조



[representative instance of the RAG process applied to question answering [1]]

Naive RAG의 basic pipeline

Indexing → Retrieval → Generation

3.1 Indexing

document를 search 가능한 형태로 준비하는 단계(user query가 들어오기 전에 미리 수행)

Raw documents
→ chunking
→ embedding
→ vector database 저장

- **chunk**
 - 원본 document를 처리 가능한 단위로 나누기 위해서 사용
- **embedding**

- text의 semantic meaning을 numerical vector로 represent하기 위해서 사용
- **vector database**
 - embedding vector를 저장
 - similarity search를 할 수 있는 database

3.2 Retrieval

user query와 semantically relevant한 chunk를 찾는 단계

- LLM이 답변을 생성할 때 참고할 evidence candidate를 찾는 것

User query

→ query embedding 생성

→ vector database에서 semantic similarity 기반 search

→ top-k chunks 반환

- **semantic similarity**
 - word가 exact match되는지보다 meaning이 얼마나 similar한지를 보는 기준
 - “휴가”와 “연차” 사이의 semantic similarity

3.3 Generation

retrieved chunks와 user query를 함께 LLM input에 include하여 답변을 생성하는 단계

User query + retrieved chunks

→ prompt/context 구성

→ LLM input

→ answer generation

LLM이 intrinsic knowledge만으로 answer하는 것이 아니라, retrieved external chunks를 evidence로 사용해 answer하도록 만드는 방식

4. RAG의 장점

1. **latest information 반영**
 - model을 retrain하지 않아도 external documents를 update하면 최신 정보 활용 가능
2. **domain-specific knowledge 활용**
 - LLM이 원래 알지 못하는 data를 external DB를 통해 사용
 - 특정 도메인/분야/문서 등의 전문 지식의 답변 근거를 DB에서 찾을 수 있음
3. **hallucination 완화**

- LLM이 retrieved evidence를 참고
- 완전한 극복은 하지 못함(retrieved evidence를 찾는 것을 실패할 수 있기 때문에)

4. source traceability

- 사용된 retrieved chunks나 source를 함께 제공
- 근거 추적을 쉽게 할 수 있음

5. retraining cost

- 모든 knowledge를 model parameter에 넣지 않음
- external knowledge base를 활용하기 때문에 retraining cost 감소

5. RAG의 한계

retrieval quality에 크게 의존

Problem	Description
DB에 information이 없음	external knowledge에 답변 근거가 없음
DB에는 있는데 retrieve하지 못함	retriever가 relevant chunk를 찾지 못함
irrelevant chunk를 retrieve함	질문과 맞지 않는 chunk가 LLM input에 들어감
partial information만 retrieve함	답변이 불충분하거나 과도한 inference가 생길 수 있음
noisy chunks가 많음	중요한 정보가 희석되거나 confuse 생김

Retrieval이 실패하면 Generation 단계에서 LLM은 부족하거나 잘못된 evidence로 답변 생성

이때 LLM이 답변을 생성할 때

- fail을 감지하고 답변을 안 하게 할 수도 있고
- intrinsic knowledge나 guess로 답변을 생성할 수도 있는데, hallucination 가능성 존재

Retrieval failure

- insufficient / wrong evidence
- poor generation quality
- hallucination 가능성 증가

6. RAG를 쓰는 case / 쓰지 않는 case

6.1 RAG를 쓰기 좋은 case

- 최신 정보 필요한 경우
- specific documents 기반 답변이 필요한 경우
 - 회사 내부 규정, 의료 도메인, 법률 전문 지식 등
- source나 evidence를 함께 provide해야 하는 경우(근거 추적이 필요한 경우)
- model이 training data에서 보지 못한 private data를 활용해야 하는 경우
- data가 자주 update되어 model fine-tuning 비용이 큰 경우

6.2 RAG가 꼭 필요하지 않은 case

- LLM intrinsic knowledge만으로 충분한 경우(범용 지식, 이미 많이 학습한 데이터에 대한 질문)
 - creative generation이나 general reasoning이 더 중요한 경우
-

7. RAG 사용 시 고려사항

answer quality에 영향을 미치는 것들

- document를 어떻게 chunking?(chunk size)
 - chunk가 크면 불필요한 정보 많이 섞임
 - 작으면 context 잘리는 문제
- 어떤 embedding model을 사용?
 - user query와 document의 semantic similarity를 잘 표현하는 모델을 사용해야
 - 관련있는 chunk를 잘 검색할 수 있음
- retrieved chunks를 reranking?
 - 최초 retrieval 결과에 관련성 낮은 chunk가 섞일 수 있음
 - 왜?
 - 항상 query 의도와 정확하게 맞는 chunk 고르기 어려움
 - 비슷한 단어가 많은데 실제 answer에 대한 근거는 아닌 chunk가 top-k에 포함될 수 있음
 - 더 중요한 chunk를 위로 올리는 과정 필요
 - 어떻게?
 - retrieved chunks를 re-scoring 해서 더 중요한 chunk를 갱신
 - cross-encoder, ColBERT, LLM-based reranking
- irrelevant chunk를 filter하거나 compress할 것인가?
 - 관련 없는/중복 정보 처리해서 답변 잘 생성하기 위해
 - retrieved chunks 전체를 넣는거 보다, answer에 필요한 부분만 filter, compress 해서 context quality를 높임
- evidence가 부족할 때 처리

- 근거 부족한데, LLM이 추론해서 답변 생성하면 hallucination 가능성 있으니
- fallback 필요할 수도 있음. "답변을 생성하기 위한 근거가 부족합니다..."
- answer에 source를 함께 제공?
 - reliability 필요할 때, evidence tracing 기능 제공할 때 필요
- RAG system을 어떻게 평가할 때 어떤 것들이 중요?
 - **retrieval quality**: query와 relevant한 chunk를 잘 검색했나?
 - **context relevance**: retrieved context가 질문에 실제로 필요한 정보?
 - **faithfulness**: 답변이 retrieved evidence에 기반해 생성됨?
 - **answer quality**: 답변이 질문 의도에 맞게 생성?
 - **source accuracy**: 제공한 source가 실제 답변 근거와 일치?

8. RAG 사용하는 방법

1. RAG pipeline self-build

```

Documents
→ preprocessing
→ chunking
→ embedding
→ vector database 저장
→ user query embedding
→ top-k retrieval
→ reranking / filtering
→ prompt construction
→ LLM answer generation
→ source 제공
  
```

필요한 component

Component	Description
document loader	documents load (PDF, Markdown, HTML)
chunker	document를 chunk 단위로 split
embedding model	chunk와 query를 semantic vector로 convert
vector database	embedding vector를 store하고 similarity search 수행
retriever	query와 relevant한 chunks를 retrieve
reranker	retrieved chunks를 다시 score해서 relevance가 높은 순서로 reorder
LLM	retrieved context를 기반으로 답변 생성

Component	Description
evaluator	retrieval quality, faithfulness, answer quality를 evaluate

open-source framework

직접 구현하는 방식 말고 만들어진 라이브러리 가져다 쓰는 방법도 존재
LangChain, LlamaIndex: 둘 다 파이썬에서 사용 가능

2. platform/service 사용

2.1 LLM provider built-in RAG

LLM provider가 file upload, vector store, retrieval tool을 함께 제공하는 방식

Service	Description	Suitable case
OpenAI File Search	uploaded files를 자동으로 parse, chunk, embed하고 vector/keyword search로 relevant content를 retrieve하는 tool	OpenAI API 기반으로 빠르게 document QA를 만들 때
Amazon Bedrock Knowledge Bases	data source를 연결하면 ingestion, chunking, embedding, vector store 저장, retrieval, prompt augmentation을 관리해주는 AWS managed RAG service	AWS 환경에서 enterprise RAG를 만들 때
Google Vertex AI / Gemini Enterprise Agent Platform	Google Cloud 기반으로 enterprise search, agent, data source integration을 제공하는 managed AI platform	GCP 환경에서 data-connected agent/RAG를 만들 때

2.2 Vector database service

RAG의 retrieval layer를 위해 vector database만 따로 사용하는 방식이다.
이 경우 chunking, embedding, prompt construction, LLM 연결은 직접 구현하거나
LangChain/LlamaIndex 같은 framework와 함께 구성

Service	Description
Pinecone	managed vector database service
Weaviate	vector database + hybrid search 지원
Milvus / Zilliz	open-source 또는 managed vector database

Service	Description
Qdrant	vector similarity search engine
Chroma	local/prototype용으로 자주 쓰는 vector database
Elasticsearch / OpenSearch vector search	기존 search engine에 vector search 기능을 결합

- RAG pipeline을 직접 control
- vector search infrastructure은 제공된 서비스 이용

2.3 No-code / low-code RAG platform

UI 기반으로 knowledge base를 만들고, chatbot이나 workflow에 연결하는 방식

Platform	Description	Suitable case
Dify	knowledge base, knowledge retrieval node, workflow/chatflow를 통해 RAG application을 만들 수 있는 platform	개인/팀 단위로 빠르게 RAG app을 만들 때
Chatbase	PDF, docs, website content 등을 upload해서 chatbot을 만들 수 있는 service	고객지원/FAQ chatbot을 빠르게 만들 때
Pinecone Canopy	Pinecone 기반 RAG application framework	Pinecone vector DB 기반으로 RAG app을 구성할 때

Memo

- RAG의 3-step은 Indexing → Retrieval → Generation
 - 최신 RAG은 구조는 기본 3-step을 따르는데 검색 품질, 답변의 신뢰성을 높이는 방식으로 발전
- 벡터 유사도가가깝다고 무조건 정답은 아니다
 - 검색 엔진은 '유사도가 높은' 단어를 찾음
 - 사용자는 '문제를 해결할 정보' 단어를 원함
 - keyword 검색과 vector 검색을 이용한 Hybrid search 필요
- vector DB에서 최신 정보가 업데이트 되었을 때, RAG에 반영하려면 어떻게 설계?
 - full indexing 하면 cost 기하급수적으로 늘어남. incremental indexing 사용
 - CDC(Change Data Capture): DB에서 변경 로그 추적하여 변경된 데이터만 vector DB로 보냄

- Hashing: 각 문서의 해시값 저장하고, 새로운 문서와 비교해서 바뀐 경우만 업데이트
 - Upsert: DB에서 기존 내용이 있으면 update, 없으면 insert
 - freshness 관리 전략
 - Lambda Architecture
 - Batch Layer(과거 데이터를 고품질로 인덱싱), Speed Layer(스트리밍 데이터를 즉시 반영)
 - Small to Big Retrieval: 소량 데이터는 fast한 inmemory DB(Redis...)에 저장
 - 검색할 때, vector DB와 병렬 조회
 - Metadata Filtering: ts(timestamp) 부여해서, 정해진 기간 데이터 우선 적용
-

Links

[1] Y. Gao et al, "Retrieval-Augmented Generation for Large Language Models: A Survey", Shanghai Research Institute for Intelligent Autonomous Systems, Tongji University, March 2024, <https://arxiv.org/pdf/2312.10997>